

Tarea 3: Procesamiento Streaming de métricas con Apache Spark y visualización con Elasticsearch

Profesor: Nicolás Hidalgo

Ayudantes: Isidora González, Natalia Ortega, Joaquín Villegas, Vicente Díaz y Benjamín Aceituno

Introducción y Contexto

En la primera entrega se implementó una plataforma distribuida para el análisis de consultas geoespaciales utilizando el dataset Google Open Buildings, incorporando un mecanismo de caché para optimizar consultas repetidas, en la segunda entrega se evolucionó la arquitectura incorporando Apache Kafka como sistema de colas de mensajes, implementando mecanismos de reintento y Dead Letter Queue para evitar la pérdida de consultas durante fallas temporales.

El sistema actual, tal como fue entregado en la Tarea 2, es capaz de procesar consultas en tiempo real, manejar picos de tráfico y reintentar consultas fallidas, sin embargo, todas las métricas generadas por el sistema (throughput, latencias, hit rates, reintentos) solo se registran en archivos de log o en memoria, lo que dificulta el monitoreo en tiempo real del estado del sistema.

En esta tercera entrega se propone incorporar un pipeline de procesamiento streaming de métricas utilizando Apache Spark Structured Streaming, junto con un stack de visualización compuesto por Elasticsearch y Kibana, el objetivo principal será consumir las métricas generadas por el sistema de la Tarea 2 desde un tópico de Kafka, procesarlas en ventanas de tiempo para calcular agregaciones como throughput promedio, latencia p95 y hit rate por minuto, almacenar estos resultados en Elasticsearch y finalmente visualizarlos a través de dashboards interactivos en Kibana.

Metodología

La tercera entrega estará enfocada en el procesamiento streaming de métricas y su visualización, el sistema deberá reutilizar todos los servicios desarrollados en la Tarea 2, incluyendo el Generador de Tráfico, el Sistema Caché, el Generador de Respuestas, el Sistema de Métricas base, y los componentes de Kafka.

Se deberán realizar las siguientes adiciones y modificaciones:

- El Sistema de Métricas deberá ser extendido para publicar cada evento de métrica en un tópico de Kafka dedicado llamado metrics-topic.
- Se deberá implementar un job de Apache Spark Structured Streaming que consuma desde metrics-topic, procese los datos en ventanas de tiempo y escriba los resultados agregados en Elasticsearch.
- Se deberá desplegar un stack de Elasticsearch y Kibana, configurando Elasticsearch como almacén de los resultados procesados por Spark y Kibana como herramienta de visualización.
- Se deberán construir dashboards en Kibana que muestren las métricas agregadas.

La evaluación de esta etapa se centrará principalmente en:

1. Publicación de métricas desde el sistema Tarea 2 hacia un tópico dedicado de Kafka.
2. Implementación de un job de Spark Structured Streaming con ventanas de tiempo.
3. Integración de Spark con Elasticsearch para el almacenamiento de resultados agregados.
4. Construcción de dashboards interactivos en Kibana para visualizar las métricas.
5. Evaluación de la latencia de visualización y la capacidad de reflejar fallas temporales.

1. Descripción del Sistema

La arquitectura del sistema deberá mantenerse modular y desacoplada, extendiendo la arquitectura de la Tarea 2 con nuevos componentes de procesamiento streaming y visualización. El sistema completo deberá considerar los siguientes componentes:

1. **Generador de Tráfico:** Genera consultas Q1-Q5 utilizando las distribuciones Zipf y uniforme implementadas en la Tarea 1 y 2.
2. **Kafka Producer y Tópicos Principales:** Publica las consultas generadas en un tópico principal de Kafka, se mantienen desde la Tarea 2.
3. **Sistema Caché:** Almacena respuestas previamente calculadas utilizando Redis, se mantiene la misma configuración de tamaño, TTL y política de remoción definida en la Tarea 2.
4. **Consumidores Kafka:** Consumen consultas desde el tópico principal e interactúan con el sistema caché, derivando al Generador de Respuestas en caso de cache miss.
5. **Generador de Respuestas:** Procesa las consultas delegadas por los consumidores cuando no existe información en caché.
6. **Sistema de Métricas (mejorado):** Componente encargado de registrar cada consulta procesada y publicar un evento estructurado en el tópico `metrics-topic`. Cada evento deberá contener información como timestamp, tipo de consulta, latencia individual, resultado de la búsqueda en caché, cantidad de reintentos y estado final de la consulta. Este componente no necesita calcular métricas agregadas, ya que dicho procesamiento será realizado posteriormente por Spark Structured Streaming.
7. **Apache Spark Structured Streaming:** Este es el componente central nuevo, consiste en un job de PySpark que se ejecuta de forma continua y realiza las siguientes operaciones: lee el stream desde `metrics-topic`, parsea los mensajes JSON, aplica ventanas de tiempo deslizantes con actualización, calcula dentro de cada ventana el throughput (cantidad de consultas exitosas por minuto), la latencia p50 y p95, el hit rate (proporción de cache hits), y el retry rate (proporción de consultas con reintentos), y finalmente escribe los resultados agregados en Elasticsearch.
8. **Elasticsearch:** Almacena los resultados agregados producidos por Spark, indexándolos para que sean consultables por Kibana.
9. **Kibana:** Herramienta de visualización que se conecta a Elasticsearch y permite construir dashboards interactivos, debe mostrar visualizaciones de las métricas procesadas por Spark, incluyendo al menos throughput, latencias p50/p95, hit rate y retry rate..

2. Flujo del Sistema

El flujo del sistema tiene dos partes que funcionan al mismo tiempo: el procesamiento de consultas y el procesamiento de métricas.

Procesamiento de consultas (utilizado en la Tarea 2)

1. El Generador de Tráfico envía consultas al tópico principal de Kafka.
2. Los consumidores toman las consultas y revisan el caché:
 - Si la consulta está en caché, se devuelve la respuesta al instante.
 - Si no está en caché, se envía al Generador de Respuestas.
3. El Generador de Respuestas procesa la consulta y entrega el resultado.

Procesamiento de métricas (a implementar ahora)

1. El Sistema de Métricas mejorado publica cada evento en un tópico de Kafka llamado `metrics-topic`. Cada mensaje incluye: timestamp, tipo de consulta, latencia, si fue cache hit o miss, y cantidad de reintentos.
2. Spark lee los mensajes de `metrics-topic` y los agrupa en ventanas de tiempo, actualizando los cálculos cada cierto tiempo. Para cada ventana, Spark calcula:
 - Throughput: cuántas consultas exitosas por minuto.

- Latencia p50 y p95: cuánto demoran las consultas.
- Hit rate: qué porcentaje de consultas encontró respuesta en caché.
- Retry rate: qué porcentaje de consultas necesitó reintentos.

Luego, Spark guarda estos resultados en Elasticsearch.

3. Kibana lee los resultados desde Elasticsearch y los muestra en dashboards con gráficos que se actualizan automáticamente.

De esta forma, el sistema sigue procesando consultas igual que en la Tarea 2, pero ahora se puede ver su comportamiento en tiempo real a través de los gráficos de Kibana.

3. Escenarios de Evaluación

El sistema de monitoreo deberá evaluarse utilizando los escenarios experimentales implementados en la Tarea 2. En particular, se espera analizar cómo el pipeline de observabilidad refleja el comportamiento del sistema bajo las siguientes condiciones:

1. Operación normal.
2. Procesamiento con uno y múltiples consumidores Kafka.
3. Falla temporal del Generador de Respuestas.
4. Consultas con reintentos y uso de Dead Letter Queue.
5. Escenarios de alta carga y *spikes* de tráfico.

La evaluación deberá centrarse en verificar que las métricas agregadas y los dashboards construidos permiten identificar y caracterizar correctamente estos escenarios.

Análisis y Discusión

En esta sección se debe realizar un análisis crítico y una discusión fundamentada sobre el sistema de monitoreo implementado y la utilidad de las métricas obtenidas. El análisis debe estar respaldado por los datos recolectados y las visualizaciones generadas en Kibana. Se deben abordar obligatoriamente los siguientes puntos:

1. Visualización y análisis de métricas.

Presentar y explicar los dashboards construidos en Kibana. Se deberán incluir capturas de las visualizaciones más relevantes (por ejemplo, throughput, latencia p50/p95, hit rate, retry rate y backlog, si este último fue implementado).

Para cada dashboard se deberá justificar:

- Qué información entrega.
- Qué tipo de comportamiento del sistema permite observar.
- Por qué dicha métrica resulta útil para monitorear una arquitectura distribuida.

2. Análisis de escenarios de ejecución.

Utilizando los escenarios experimentales desarrollados en la Tarea 2 (operación normal, múltiples consumidores, fallas temporales, reintentos y spikes de tráfico), se deberá analizar cómo estos eventos se reflejan en las métricas visualizadas.

Se espera discutir preguntas como:

- ¿Es posible identificar una falla temporal únicamente observando los dashboards?
- ¿Cómo se manifiesta un aumento repentino de carga en las métricas?
- ¿Qué indicadores permiten detectar la existencia de reintentos o degradación del rendimiento?

3. Justificación de la arquitectura de monitoreo.

Justificar el uso de Apache Spark Structured Streaming para el procesamiento de métricas en tiempo real y la elección de Elasticsearch y Kibana como plataforma de almacenamiento y visualización.

La discusión debe considerar aspectos como:

- Procesamiento continuo mediante ventanas temporales.
- Separación entre el plano de procesamiento y el plano de observabilidad.
- Ventajas de utilizar un pipeline desacoplado para el monitoreo del sistema.
- Desafíos encontrados durante la integración entre Kafka, Spark y Elasticsearch.

4. **Discusión: monitoreo en tiempo real versus análisis posterior.**

Reflexionar sobre las ventajas y limitaciones de incorporar una arquitectura de observabilidad en tiempo real frente a depender únicamente de archivos de log o métricas registradas localmente.

En particular, discutir:

- ¿Qué beneficios aporta el monitoreo en línea para la detección de fallos?
- ¿Qué tipo de problemas pueden identificarse rápidamente mediante dashboards interactivos?
- ¿Qué limitaciones presenta este enfoque y qué mejoras podrían incorporarse en futuras versiones del sistema?

Requisitos de la entrega

La evaluación de este hito se basará en los siguientes entregables:

- **Video de Demostración:** Video de aproximadamente 15 minutos mostrando el pipeline de métricas en funcionamiento, la actualización de las visualizaciones y cómo los dashboards reflejan distintos escenarios experimentales, junto a su respectivo análisis y discusión.
- **Código Fuente:** Repositorio público en GitHub o GitLab con el código completo del proyecto. Se espera que se siga el método de trabajo en GitHub exigido en la Tarea 2.
- **Archivos de Despliegue:** Archivo docker-compose.yml y/o Dockerfile necesarios para ejecutar el sistema.

Reglas y Consideraciones

- **Fecha de Entrega:** La entrega se realiza el día 21 de junio vía Canvas del curso.
- **Integrantes:** La tarea debe ser realizada en grupos de hasta 2 alumnos/as.
- **Tecnología:** Se requiere Docker y Apache Kafka. El stack interno queda a elección del equipo, siempre que cada decisión esté correctamente justificada.
- **Ética y Autoría:** Se debe respetar el reglamento de la universidad respecto a plagio y autoría.